AKADEMIA NAUK STOSOWANYCH
W NOWYM SĄCZU

Wydział Nauk Inżynieryjnych
Katedra Informatyki

DOKUMENTACJA PROJEKTOWA

ZAAWANSOWANE PROGRAMOWANIE

Algorytm listy dwukierunkowej z zastosowaniem GitHub

Autor:
Kupchynskyi Oleksandr

Prowadzący:
mgr inż. Dawid Kotlarski

Nowy Sącz 2026

Spis treści

1. Ogólne określenie wymagań **4**

2. Analiza problemu **5**

2.1. 5

2.2. Zastosowanie 5

2.3. Sposób działania 5

2.4. Wzorce Factory i Iterator 6

2.5. Git, Github, Doxygen 6

3. Projektowanie **8**

3.1. Założenia 8

3.2. Klasy 8

3.3. Node 8

3.4. Node Fabryka 9

3.5. Iterator 9

3.6. ListaDwukierunkowa 9

3.7. Podsumowanie 9

4. Implementacja **10**

4.1. Implementacja Node.h 10

4.2. Implementacja Node Fabryka.h 10

4.3. Implementacja Node Fabryka.cpp 11

4.4. Implementacja Iterator.h 11

4.5. Implementacja Iterator.cpp 13

4.6. Implementacja ListaDwukierunkowa.h 14

4.7. Implementacja ListaDwukierunkowa.cpp 16

4.8. Implementacja main.cpp 22

5. Wnioski **24**

Literatura **25**

Spis rysunków **25**

Spis tabel **26**

Spis listingów **27**

1. Ogólne określenie wymagań

Celem projektu jest stworzenie listy dwukierunkowej w języku c++ używając między innymi wzorców projektowych takich jak Factory i Iterator. Wzorzec Factory będzie odpowiedzialny za tworzenie węzłów, oddzielając proces ich inicjalizacji od logiki zarządzającej listą. Z kolei Iterator umożliwi uporządkowane przemieszczanie się pomiędzy elementami bez konieczności bezpośredniego odwoływania się do wewnętrznej struktury danych. Zastosowanie tych rozwiązań powinno wpłynąć na przejrzystość kodu, jego organizację oraz możliwość dalszej rozbudowy programu.

Lista dwukierunkowa to rodzaj listy łańcuchowej, gdzie każdy węzeł oprócz przechowywania danych i wskaźnika na następny węzeł, posiada również wskaźnik na poprzedni węzeł. Daje to możliwość poruszania się w obu kierunkach na liście, co umożliwia dodatkowe operacje, które nie są możliwe w listach jednokierunkowych.

Projekt ma posiadać takie elementy :

- dodawanie nowych elementów na początku i na końcu listy,
- wstawianie elementu pod określonym indeksem,
- usuwanie pierwszego oraz ostatniego elementu,
- usuwanie elementu znajdującego się pod wskazanym indeksem,
- wyświetlanie wszystkich elementów w kolejności od początku do końca,
- prezentowanie zawartości listy w odwrotnej kolejności,
- przechodzenie do następnego i poprzedniego elementu,
- usuwanie wszystkich elementów i całkowite czyszczenie listy.

Kod należy podzielić na klasy umieszczone w oddzielnych plikach nagłówkowych i źródłowych. Taki sposób organizacji ułatwia analizowanie poszczególnych części programu, wprowadzanie modyfikacji oraz testowanie zaimplementowanych funkcjonalności.

Weryfikacja poprawności działania programu powinna zostać przeprowadzona w funkcji main(). Testy muszą obejmować wszystkie dostępne metody, zarówno w typowych sytuacjach, jak i w przypadkach szczególnych. Należy sprawdzić działanie listy pustej, listy zawierającej jeden element oraz struktury składającej się z wielu węzłów. Ważne jest również upewnienie się, że po operacjach dodawania i usuwania połączenia pomiędzy elementami pozostają prawidłowe, a wyświetlanie danych w obu kierunkach przebiega zgodnie z założeniami.

2. Analiza problemu

2.1.

Celem analizy jest określenie sposobu przechowywania i modyfikowania danych w liście dwukierunkowej. Każdy węzeł zawiera wartość i dwa wskaźniki opisujące jego sąsiedztwo. Pierwszy element nie ma poprzednika, a ostatni nie wskazuje następnego węzła. Pusta lista nie zawiera żadnych obiektów. Elementy są alokowane dynamicznie, więc nie muszą zajmować sąsiadujących komórek pamięci. Pozwala to dodawać i usuwać węzły bez przesuwania pozostałych danych. Wymaga jednak zachowania spójnych połączeń: błędna aktualizacja wskaźników może przerwać przechodzenie po liście lub doprowadzić do niepoprawnego dostępu do pamięci.

2.2. Zastosowanie

Lista dwukierunkowa jest przydatna tam, gdzie kolejność danych może się zmieniać, a użytkownik powinien mieć możliwość przejścia zarówno do przodu, jak i do tyłu. Wstawianie i usuwanie węzłów nie wymaga przesuwania całego zbioru, jeśli znane jest miejsce modyfikacji. Przykładowe zastosowania obejmują:

- historię przeglądarki, w której można wrócić do poprzedniej strony i przejść ponownie dalej
- playlistę umożliwiającą wybór kolejnego lub wcześniejszego utworu
- mechanizmy cofania i ponawiania operacji w programach
- kolekcje elementów interfejsu, których kolejność często ulega zmianie

Nie w każdej sytuacji lista będzie najlepszym wyborem. Odczyt elementu o danym indeksie wymaga przejścia przez wcześniejsze węzły, podczas gdy tablica zapewnia bezpośredni dostęp. Strukturę należy więc dobierać do rodzaju najczęściej wykonywanych operacji.

2.3. Sposób działania

Program będzie przechowywał wskaźniki do początku i końca listy, na przykład glowa i ogon. Każdy węzeł będzie wskazywał sąsiadów. Po każdej operacji modyfikującej dane trzeba sprawdzić, czy wszystkie powiązania oraz informacje o skrajnych elementach są aktualne. Dodawanie: nowy węzeł na początku zostaje połączony z

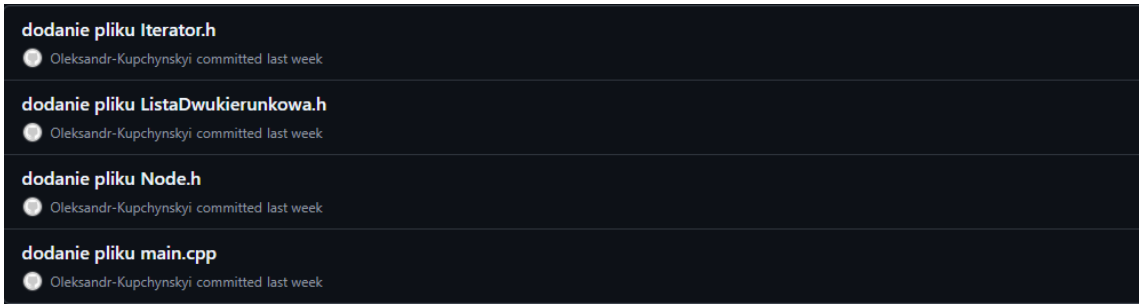
dotychczasową głową, a przy dopisywaniu na końcu — z poprzednim ogonem. Wstawianie wewnątrz listy wymaga znalezienia pozycji oraz ustawienia wskaźników nowego węzła i jego sąsiadów. Usuwanie: przy kasowaniu pierwszego lub ostatniego elementu należy przesunąć odpowiedni wskaźnik. Jeśli usuwany jest węzeł ze środka, jego poprzednik i następnik powinny zostać połączeni bezpośrednio. Po odłączeniu obiektu należy zwolnić zajętą przez niego pamięć. Przeglądanie: wypisywanie w zwykłej kolejności rozpoczyna się od początku i korzysta ze wskaźnika następnego węzła. Wyświetlanie od końca wykorzystuje wskaźnik poprzednika. Iterator powinien sprawdzać, czy przejście jest możliwe, zanim odwoła się do kolejnego elementu. Czyszczenie: wszystkie węzły są kolejno usuwane, a pamięć zwalniana. Po zakończeniu zarówno początek, jak i koniec listy powinny oznaczać brak elementów. Należy szczególnie uważać na przypadek pustej listy i listy zawierającej tylko jeden węzeł.

2.4. Wzorce Factory i Iterator

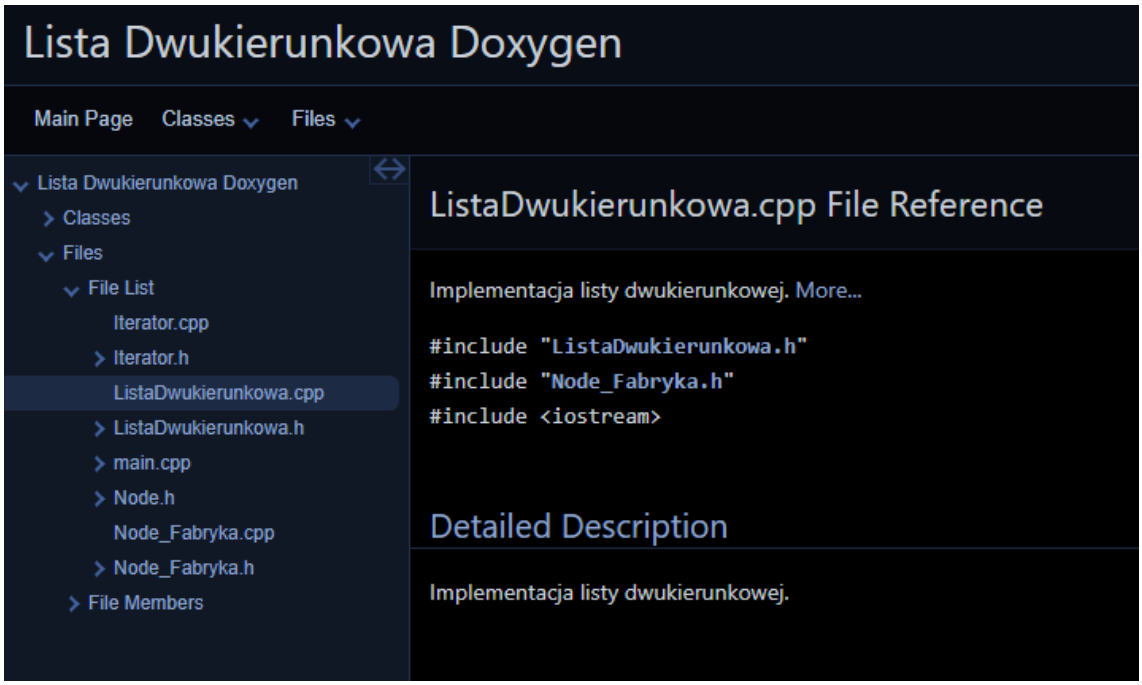
Factory oddziela tworzenie obiektów od kodu, który z nich korzysta. W tym projekcie klasa fabryki będzie przygotowywać nowe węzły na podstawie otrzymanej wartości. Dzięki temu logika listy nie musi powielać instrukcji tworzących elementy. Fabryka nie zastępuje jednak obowiązku poprawnego usuwania węzłów i zwalniania pamięci. Iterator zapewni sposób przechodzenia po kolekcji bez konieczności poznawania całej jej wewnętrznej implementacji. Będzie przechowywał bieżącą pozycję i pozwalał przejść do poprzedniego lub następnego węzła. Powinien też uwzględniać granice listy oraz sytuację, gdy struktura jest pusta. Zastosowanie obu wzorców rozdziela trzy zadania: tworzenie elementów, nawigację po nich i zarządzanie zawartością listy.

2.5. Git, Github, Doxygen

Git służy do zapisywania historii zmian w projekcie. Poszczególne commity pozwalają porównywać kolejne wersje kodu i w razie potrzeby odtworzyć wcześniejszy stan. GitHub umożliwia przechowywanie repozytorium w sieci oraz synchronizowanie pracy między różnymi kopiami projektu. Wysyłanie zmian i ich pobieranie pozwala utrzymywać aktualną wersję kodu w kilku lokalizacjach. Doxygen tworzy dokumentację na podstawie komentarzy umieszczonych w kodzie. Może opisywać klasy, metody, parametry i wartości zwracane. Aby z niego skorzystać, należy odpowiednio opisać kod i skonfigurować katalogi wejściowe oraz wyjściowe, nazwę projektu i format dokumentacji.



Rys. 2.1. Comity na Github



Rys. 2.2. HTML doxygen

3. Projektowanie

3.1. Założenia

Program zostanie zorganizowany w języku C++ z wykorzystaniem kilku klas, z których każda będzie realizowała odrębną część zadania. Dane będą przechowywane w dynamicznie tworzonych węzłach, a wskaźniki początku i końca umożliwią szybkie dotarcie do skrajnych elementów. Projekt obejmuje dodawanie, wstawianie, usuwanie, wyświetlanie i czyszczenie listy. Każdy węzeł będzie połączony z poprzednim i następnym, co pozwoli nawigować w obu kierunkach. Istotne będzie poprawne aktualizowanie wskaźników, zwłaszcza gdy operacja dotyczy pierwszego, ostatniego lub jedynego elementu. Za tworzenie węzłów odpowie fabryka, a za przechodzenie między nimi iterator. Rozdzielenie tych funkcji od metod zarządzających listą ma ograniczyć powtarzanie kodu i ułatwić jego utrzymanie.

3.2. Klasy

Architektura programu będzie oparta na klasach Element, Fabryka, Iterator i ListaDwukierunkowa. Rozdzielenie obowiązków ma ułatwić testowanie poszczególnych części oraz ograniczyć zależności między nimi. Element będzie opisywał pojedynczy węzeł i przechowywał wartość wraz ze wskaźnikami do obu sąsiadów. Fabryka zajmie się konstruowaniem nowych węzłów. Dzięki temu operacje listy będą mogły korzystać ze wspólnego sposobu tworzenia obiektów. Iterator będzie wskazywał aktualny element i udostępniał przejścia w obu kierunkach. Należy określić jego zachowanie przy osiągnięciu granicy listy. ListaDwukierunkowa będzie przechowywać wskaźniki początku i końca oraz udostępniać metody modyfikowania, wyświetlania i opróżniania struktury. W razie potrzeby skorzysta z fabryki i iteratora. Taki podział pozwoli zmieniać sposób tworzenia węzłów lub nawigacji bez konieczności przebudowy całej klasy zarządzającej listą.

3.3. Node

Klasa będzie reprezentowała pojedynczą wartość w strukturze i przechowywała odwołania do sąsiednich węzłów. Dynamiczne tworzenie obiektów umożliwi zmianę rozmiaru listy w czasie pracy programu. Wskaźniki muszą być prawidłowo ustawiane przy każdej zmianie zawartości.

3.4. Node Fabryka

Zadaniem fabryki będzie utworzenie węzła dla przekazanej wartości i zwrócenie go do dalszego wykorzystania. Dzięki temu kod odpowiedzialny za konstrukcję obiektów znajdzie się w jednym miejscu, zamiast pojawiać się w wielu metodach listy. Niezależnie od miejsca, w którym nowy element zostanie umieszczony, będzie tworzony przy użyciu tego samego mechanizmu. Uprości to utrzymanie projektu i ewentualne zmiany sposobu konstruowania węzłów.

3.5. Iterator

Iterator będzie przechowywał wskaźnik do aktualnego węzła i umożliwiał przejście do elementu poprzedniego albo następnego. Skorzysta przy tym z powiązań zapisanych w węzłach. Należy zabezpieczyć go przed próbą przejścia poza początek lub koniec listy. Trzeba też uwzględnić usunięcie elementu, na który wskazywał iterator, ponieważ dalsze korzystanie z takiego wskaźnika byłoby niebezpieczne.

3.6. ListaDwukierunkowa

Klasa ListaDwukierunkowa będzie zarządzać całą strukturą. Przechowa wskaźniki do skrajnych węzłów i udostępni operacje dodawania, wstawiania, usuwania, wyświetlania oraz czyszczenia. Będzie także odpowiadać za utrzymanie prawidłowych połączeń i zwalnianie pamięci. Dodanie elementu na początku polega na połączeniu nowego węzła z dotychczasową głową i aktualizacji wskaźnika początku. Analogicznie przy dodawaniu na końcu aktualizowany jest ogon. Jeśli lista nie zawierała wcześniej żadnych elementów, nowy węzeł staje się jednocześnie początkiem i końcem. Wstawienie pod indeksem wymaga odnalezienia właściwej pozycji i połączenia nowego węzła z sąsiadami. Usuwanie wymaga odłączenia wybranego elementu, poprawienia wskaźników i zwolnienia pamięci. Operacja czyszczenia powinna usunąć wszystkie węzły i pozostawić obiekt w stanie umożliwiającym ponowne użycie.

3.7. Podsumowanie

Projekt opiera się na rozdzieleniu czterech zadań: reprezentacji węzła, tworzenia obiektów, nawigacji i zarządzania listą. Przyjęta organizacja powinna poprawić czytelność kodu oraz ułatwić jego rozwijanie.

4. Implementacja

4.1. Implementacja Node.h

```
#ifndef NODE_H
#define NODE_H

/**
 * @file Node.h
 * @brief Definicja wezla listy dwukierunkowej.
 */

 /**
  * @class Node
  * @brief Pojedynczy wezel listy dwukierunkowej.
  */

using namespace std;

class Node {

    private:
    int data;
    Node* prev;
    Node* next;

    public:
    /**
     * @brief Tworzy nowy wezel.
     * @param wart Wartosc przechowywana w wezle.
     */
    Node(int wart) : data(wart), prev(nullptr), next(nullptr){}

    friend class ListaDwukierunkowa;
    friend class Iterator;
};

#endif
```

Listing 1. Kodu pliku Node.h

4.2. Implementacja Node Fabryka.h

```
#ifndef NODE_FABRYKA_H
```

```
#define NODE_FABRYKA_H

#include "Node.h"

/**
 * @file Node_Fabryka.h
 * @brief Deklaracja fabryki welow.
 */

class Node_Fabryka {
    public:
        static Node* utworz(int wart);
};
#endif
```

Listing 2. Kodu pliku Node Fabryka.h

4.3. Implementacja Node Fabryka.cpp

```
#include "Node_Fabryka.h"
#include "Node.h"

/**
 * @brief Tworzy nowy wezel z podana wartoscia.
 */
Node* Node_Fabryka::utworz(int wart) {
    return new Node(wart);
}
```

Listing 3. Kodu pliku Node Fabryka.cpp

4.4. Implementacja Iterator.h

```
#ifndef ITERATOR_H
#define ITERATOR_H
#include <iostream>

using namespace std;

#include "Node.h"

/**
 * @file Iterator.h
 * @brief Deklaracja iteratora listy dwukierunkowej.
```

```
 */


 /**
  * @class Iterator
  * @brief Iterator umo liwiajacy przechodzenie po wez Ćach listy.
  */

class Iterator {
private:
    Node* current;

public:

    /**
     * @brief Tworzy iterator wskazuj cy na podany wezel.
     * @param start Poczatkowy wezel, domyslnie nullptr.
     */
    Iterator(Node* start = nullptr);

    /**
     * @brief Sprawdza, czy iterator wskazuje na element.
     * @return true, je Żli element istnieje.
     */
    bool istnieje() const;

    /**
     * @brief Zwraca wartosc aktualnego elementu.
     * @return Wartosc elementu.
     * @throws std::out_of_range Gdy iterator nie wskazuje na
   element.
     */
    int wartosc() const;

    /**
     * @brief Przechodzi do nastepnego elementu.
     */
    void nastepny();

    /**
     * @brief Przechodzi do poprzedniego elementu.
     */
    void poprzedni();
};

```

```
#endif
```

Listing 4. Kodu pliku Iterator.h

4.5. Implementacja Iterator.cpp

```

/**
 * @file Iterator.cpp
 * @brief Implementacja iteratora.
 */

#include "Iterator.h"
#include <stdexcept>

Iterator::Iterator(Node* start)
    : current(start) {
}

bool Iterator::istnieje() const {
    return current != nullptr;
}

int Iterator::wartosc() const {
    if (!current) {
        throw std::out_of_range(
            "Iterator nie wskazuje na element."
        );
    }

    return current->data;
}

void Iterator::nastepny() {
    if (current) {
        current = current->next;
    }
}

void Iterator::poprzedni() {
    if (current) {
        current = current->prev;
    }
```

```
}
```

Listing 5. Kodu pliku Iterator.cpp

4.6. Implementacja ListaDwukierunkowa.h

```

#ifndef LISTA_DWUKIERUNKOWA_H
#define LISTA_DWUKIERUNKOWA_H

/**
 * @file ListaDwukierunkowa.h
 * @brief Deklaracja listy dwukierunkowej.
 */

#include "Node.h"
#include "Iterator.h"

 /**
  * @class ListaDwukierunkowa
  * @brief Lista dwukierunkowa przechowujaca liczby calkowite.
  *
  * Indeksy element w zaczynaja sie od zera.
  */
class ListaDwukierunkowa {
private:
    Node* head; ///< Pierwszy wezel listy.
    Node* tail; ///< Ostatni wezel listy.

public:
    /**
     * @brief Tworzy pusta liste.
     */
    ListaDwukierunkowa();

    /**
     * @brief Usuwa wszystkie elementy listy.
     */
    ~ListaDwukierunkowa();

    // Lista zarz dza pamiecia wezlow, dlatego
    // wylaczamy niebezpieczne kopiowanie plytkie.
    ListaDwukierunkowa(const ListaDwukierunkowa&) = delete;
    ListaDwukierunkowa& operator=(
        const ListaDwukierunkowa&
```

```
40          ) = delete;
41
42      /**
43       * @brief Dodaje element na poczatek listy.
44       * @param wart Wartosc nowego elementu.
45       */
46      void dodaj_P(int wart);
47
48      /**
49       * @brief Dodaje element na koniec listy.
50       * @param wart Wartosc nowego elementu.
51       */
52      void dodaj_K(int wart);
53
54      /**
55       * @brief Wstawia element pod wskazany indeks.
56       * @param wart Wartosc nowego elementu.
57       * @param indeks Indeks od zera;
58       * @note Indeksy ujemne i wieksze od rozmiaru
59       * listy sa ignorowane.
60       */
61      void dodaj_i(int wart, int indeks);
62
63      /**
64       * @brief Usuwa pierwszy element listy.
65       */
66      void usun_P();
67
68      /**
69       * @brief Usuwa ostatni element listy.
70       */
71      void usun_K();
72
73      /**
74       * @brief Usuwa element o podanym indeksie.
75       * @param indeks Indeks elementu od zera.
76       */
77      void usun(int indeks);
78
79      /**
80       * @brief Wyswietla liste od poczatku do ko ca.
81       */
82      void wyswietl() const;
83
84      /**
```

```
85      * @brief Wyswietla liste od ko nca do poczatku.
86      */
87     void wyswietlOdwrotnie() const;
88 
89     /**
90      * @brief Wyswietla element znajdujacy sie
91      * bezpo Żrednio za podanym indeksem.
92      * @param indeks Indeks elementu bazowego.
93      */
94     void wyswietlNastepny(int indeks) const;
95 
96     /**
97      * @brief Wyswietla element znajdujacy sie
98      * bezpo Żrednio przed podanym indeksem.
99      * @param indeks Indeks elementu bazowego.
100     */
101    void wyswietlPoprzedni(int indeks) const;
102 
103    /**
104     * @brief Usuwa wszystkie elementy listy.
105     */
106    void wyczysc();
107 
108    /**
109     * @brief Zwraca iterator wskazujacy na poczatek listy.
110     */
111    Iterator poczatek() const;
112 
113    /**
114     * @brief Zwraca iterator wskazuj cy na koniec listy.
115     */
116    Iterator koniec() const;
117 };
118 
119 #endif
```

Listing 6. Kodu pliku ListaDwukierunkowa.h

4.7. Implementacja ListaDwukierunkowa.cpp

```
1 
2 /**
3  * @file ListaDwukierunkowa.cpp
4  * @brief Implementacja listy dwukierunkowej.
5  */
```

```
6
7  #include "ListaDwukierunkowa.h"
8  #include "Node_Fabryka.h"
9
10 #include <iostream>
11
12 using namespace std;
13
14 ListaDwukierunkowa::ListaDwukierunkowa()
15     : head(nullptr), tail(nullptr) {
16 }
17
18 ListaDwukierunkowa::~ListaDwukierunkowa() {
19     wyczysc();
20 }
21
22 void ListaDwukierunkowa::dodaj_P(int wart) {
23     Node* nowy = Node_Fabryka::utworz(wart);
24
25     if (head == nullptr) {
26         head = tail = nowy;
27     }
28     else {
29         nowy->next = head;
30         head->prev = nowy;
31         head = nowy;
32     }
33 }
34
35 void ListaDwukierunkowa::dodaj_K(int wart) {
36     Node* nowy = Node_Fabryka::utworz(wart);
37
38     if (tail == nullptr) {
39         head = tail = nowy;
40     }
41     else {
42         tail->next = nowy;
43         nowy->prev = tail;
44         tail = nowy;
45     }
46 }
47
48 void ListaDwukierunkowa::dodaj_i(int wart, int indeks) {
49     if (indeks < 0) {
50         return;
```

```
51      }
52
53      if (indeks == 0) {
54          dodaj_P(wart);
55          return;
56      }
57
58      Node* pom = head;
59      int i = 0;
60
61      // Szukanie elementu, przed ktorym wstawiamy nowy.
62      while (pom != nullptr && i < indeks) {
63          pom = pom->next;
64          ++i;
65      }
66
67      if (pom == nullptr) {
68          // Indeks rowny rozmiarowi oznacza koniec listy.
69          if (i == indeks) {
70              dodaj_K(wart);
71          }
72          return;
73      }
74
75      Node* nowy = Node_Fabryka::utworz(wart);
76
77      nowy->next = pom;
78      nowy->prev = pom->prev;
79
80      pom->prev->next = nowy;
81      pom->prev = nowy;
82  }
83
84  void ListaDwukierunkowa::usun_P() {
85      if (head == nullptr) {
86          return;
87      }
88
89      Node* pom = head;
90      head = head->next;
91
92      if (head != nullptr) {
93          head->prev = nullptr;
94      }
95      else {
```

```
96          tail = nullptr;
97      }
98
99      delete pom;
100 }
101
102 void ListaDwukierunkowa::usun_K() {
103     if (tail == nullptr) {
104         return;
105     }
106
107     Node* pom = tail;
108     tail = tail->prev;
109
110     if (tail != nullptr) {
111         tail->next = nullptr;
112     }
113     else {
114         head = nullptr;
115     }
116
117     delete pom;
118 }
119
120 void ListaDwukierunkowa::usun(int indeks) {
121     if (indeks < 0) {
122         return;
123     }
124
125     Node* pom = head;
126     int i = 0;
127
128     while (pom != nullptr && i < indeks) {
129         pom = pom->next;
130         ++i;
131     }
132
133     if (pom == nullptr) {
134         return;
135     }
136
137     if (pom->prev != nullptr) {
138         pom->prev->next = pom->next;
139     }
140     else {
```

```
141         head = pom->next;
142     }
143 
144     if (pom->next != nullptr) {
145         pom->next->prev = pom->prev;
146     }
147     else {
148         tail = pom->prev;
149     }
150 
151     delete pom;
152 }
153 
154 void ListaDwukierunkowa::wyswietl() const {
155     cout << "Lista od poczatku: ";
156 
157     Iterator it(head);
158 
159     while (it.istnieje()) {
160         cout << it.wartosc() << " ";
161         it.nastepny();
162     }
163 
164     cout << '\n';
165 }
166 
167 void ListaDwukierunkowa::wyswietlOdwrotnie() const {
168     cout << "Lista od konca: ";
169 
170     Iterator it(tail);
171 
172     while (it.istnieje()) {
173         cout << it.wartosc() << " ";
174         it.poprzedni();
175     }
176 
177     cout << '\n';
178 }
179 
180 void ListaDwukierunkowa::wyswietlNastepny(
181     int indeks
182 ) const {
183     if (indeks < 0) {
184         cout << "Niepoprawny indeks.\n";
185         return;
```

```
186     }
187
188     Node* pom = head;
189     int i = 0;
190
191     while (pom != nullptr && i < indeks) {
192         pom = pom->next;
193         ++i;
194     }
195
196     if (pom == nullptr || pom->next == nullptr) {
197         cout << "Brak nastepnego elementu.\n";
198         return;
199     }
200
201     cout << "Nastepny element: "
202         << pom->next->data << '\n';
203 }
204
205 void ListaDwukierunkowa::wyswietlPoprzedni(
206     int indeks
207 ) const {
208     if (indeks < 0) {
209         cout << "Niepoprawny indeks.\n";
210         return;
211     }
212
213     Node* pom = head;
214     int i = 0;
215
216     while (pom != nullptr && i < indeks) {
217         pom = pom->next;
218         ++i;
219     }
220
221     if (pom == nullptr || pom->prev == nullptr) {
222         cout << "Brak poprzedniego elementu.\n";
223         return;
224     }
225
226     cout << "Poprzedni element: "
227         << pom->prev->data << '\n';
228 }
229
230 void ListaDwukierunkowa::wyczysc() {
```

```
231     Node* pom = head;
232 
233     while (pom != nullptr) {
234         Node* nastepny = pom->next;
235         delete pom;
236         pom = nastepny;
237     }
238 
239     head = nullptr;
240     tail = nullptr;
241 }
242 
243 Iterator ListaDwukierunkowa::poczatek() const {
244     return Iterator(head);
245 }
246 
247 Iterator ListaDwukierunkowa::koniec() const {
248     return Iterator(tail);
249 }
```

Listing 7. Kodu pliku ListaDwukierunkowa.cpp

4.8. Implementacja main.cpp

```
1     #include <iostream>
2 #include "ListaDwukierunkowa.h"
3 
4 using namespace std;
5 
6 int main(){
7     ListaDwukierunkowa lista;
8 
9     cout << "Dodanie elementow" << endl;
10 
11     lista.dodaj_P(20);
12     lista.dodaj_P(2);
13     lista.dodaj_K(3);
14     lista.dodaj_i(1,4);
15     lista.dodaj_i(50, 3);
16 
17 
18     cout << "Wyswietlanie elementow" << endl;
19     lista.wyswietl();
20 
21     lista.wyswietlOdwrotnie();
```

```
22
23      lista.wyswietlNastepny(3);
24      lista.wyswietlPoprzedni(2);
25
26      cout << "Iterator" << endl;
27      Iterator i = lista.poczatek();
28
29      if (i.istnieje()) {
30          cout << "Pierwszy element: " << i.wartosc()
31              << '\n';
32
33          i.nastepny();
34
35          if (i.istnieje()) {
36              cout << "Drugi element: " << i.wartosc() << endl;
37          }
38
39          i.poprzedni();
40
41          if (i.istnieje()) {
42              cout << "Po powrocie: " << i.wartosc() << endl;
43          }
44      }
45
46      cout << "Usuwanie elementow" << endl;
47
48      lista.usun_P();
49      lista.wyswietl();
50
51      lista.usun_K();
52      lista.wyswietl();
53
54      lista.usun(1);
55      lista.wyswietl();
56
57      return 0;
58
59  }
```

Listing 8. Kodu pliku main.cpp

5. Wnioski

Celem projektu było stworzenie programu w języku C++, który umożliwia obsługę listy dwukierunkowej z wykorzystaniem pamięci dynamicznej. Zaimplementowano funkcje dodawania, wstawiania i usuwania elementów, wyświetlania zawartości listy w obu kierunkach oraz przemieszczania się pomiędzy węzłami.

Kod podzielono na osobne klasy i pliki, a do implementacji wykorzystano wzorce projektowe Factory i Iterator. Pozwoliło to uporządkować strukturę programu oraz oddzielić tworzenie węzłów od poruszania się po liście.

Poprawność działania sprawdzono za pomocą testów obejmujących różne przypadki, w tym obsługę pustej listy i usuwanie elementów. W projekcie wykorzystano również Git i GitHub do kontroli wersji oraz Doxygen do generowania dokumentacji technicznej.

Realizacja zadania pozwoliła zdobyć praktyczne doświadczenie w zakresie struktur danych, programowania obiektowego i zarządzania pamięcią, a także poznać narzędzia wspomagające tworzenie oprogramowania.

Spis rysunków

2.1. Comity na Github 7
2.2. HTML doxygen 7

Spis tabel

Spis listingów

1. Kodu pliku Node.h 10
2. Kodu pliku Node Fabryka.h 10
3. Kodu pliku Node Fabryka.cpp 11
4. Kodu pliku Iterator.h 11
5. Kodu pliku Iterator.cpp 13
6. Kodu pliku ListaDwukierunkowa.h 14
7. Kodu pliku ListaDwukierunkowa.cpp 16
8. Kodu pliku main.cpp 22